Exp 3
Implement a Generative Adversarial Network (GAN) to generate synthetic images or tabular data.

Aim:

To implement a Generative Adversarial Network (GAN) to generate synthetic handwritten digit images using the MNIST dataset.

Dataset: MNIST (for images) or Synthetic Financial Transactions

Algorithm and Program:

Step 1: Import Libraries

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
```

Step 2: Load Dataset (MNIST)

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

dataset = datasets.MNIST(root='./data', train=True, download=True,
transform=transform)
dataloader = DataLoader(dataset, batch_size=64, shuffle=True)
```

Step 3: Define Generator

```
class Generator(nn.Module):
```

```
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(100, 256),
            nn.ReLU(),
            nn.Linear(256, 512),
            nn.ReLU(),
            nn.Linear(512, 784),
            nn.Tanh()
        )

    def forward(self, x):
        return self.model(x)
```

Step 4: Define Discriminator

```
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(784, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.model(x)
```

Step 5: Initialize Models

```
generator = Generator()
discriminator = Discriminator()
```

Step 6: Loss Function & Optimizers

```
criterion = nn.BCELoss()
```

```
g_optimizer = optim.Adam(generator.parameters(), lr=0.0002)
d_optimizer = optim.Adam(discriminator.parameters(), lr=0.0002)
```

Step 7: Training the GAN

```
epochs = 5

for epoch in range(epochs):
    for real_images, _ in dataloader:

        batch_size = real_images.size(0)
        real_images = real_images.view(batch_size, -1)

        # Labels
        real_labels = torch.ones(batch_size, 1)
        fake_labels = torch.zeros(batch_size, 1)

        # ----- Train Discriminator -----
        z = torch.randn(batch_size, 100)
        fake_images = generator(z)

        real_loss = criterion(discriminator(real_images), real_labels)
        fake_loss = criterion(discriminator(fake_images.detach()),
fake_labels)

        d_loss = real_loss + fake_loss

        d_optimizer.zero_grad()
        d_loss.backward()
        d_optimizer.step()

        # ----- Train Generator -----
        z = torch.randn(batch_size, 100)
        fake_images = generator(z)

        g_loss = criterion(discriminator(fake_images), real_labels)

        g_optimizer.zero_grad()
        g_loss.backward()
        g_optimizer.step()
```

```
    print(f"Epoch [{epoch+1}/{epochs}]  D Loss: {d_loss.item()}  G Loss:
{g_loss.item()}")
```

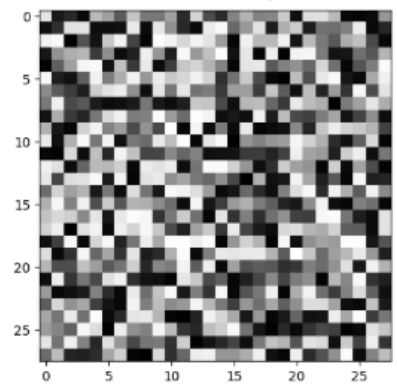
Step 8: Generate Synthetic Images

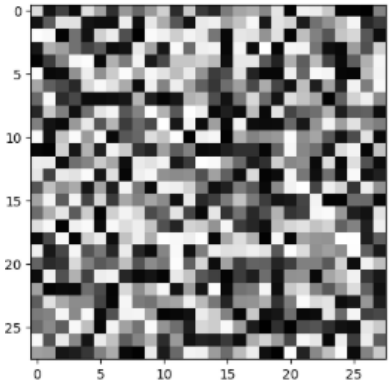
```
z = torch.randn(1, 100)
generated_image = generator(z).detach().view(28, 28)

plt.imshow(generated_image, cmap='gray')
plt.title("Generated Image")
plt.show()
```

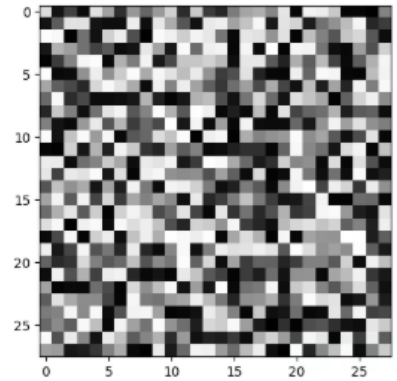
Output:

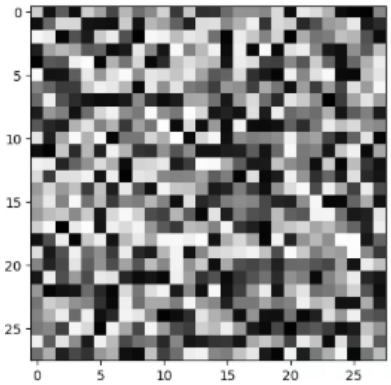
```
Epoch: 0
Epoch: 100
Epoch: 200
Epoch: 300
Epoch: 400
Epoch: 500
Epoch: 600
Epoch: 700
Epoch: 800
Epoch: 900
Epoch: 1000
Epoch: 1100
Epoch: 1200
Epoch: 1300
Epoch: 1400
Epoch: 1500
Epoch: 1600
Epoch: 1700
Epoch: 1800
Epoch: 1900
```

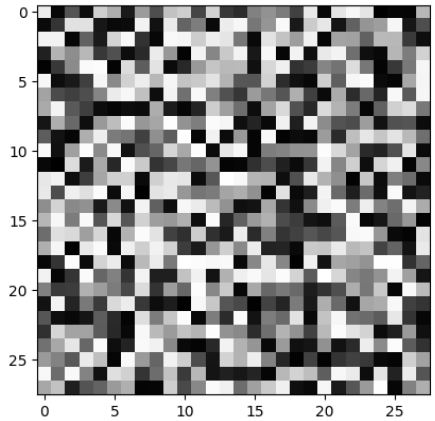
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step
```











Result:

The GAN successfully learns the distribution of handwritten digits and generates realistic synthetic images. Generator improves over epochs by fooling the discriminator, while discriminator learns to distinguish real vs fake images.